

Oracle HR Schema Subquery Analytics & Business Reporting

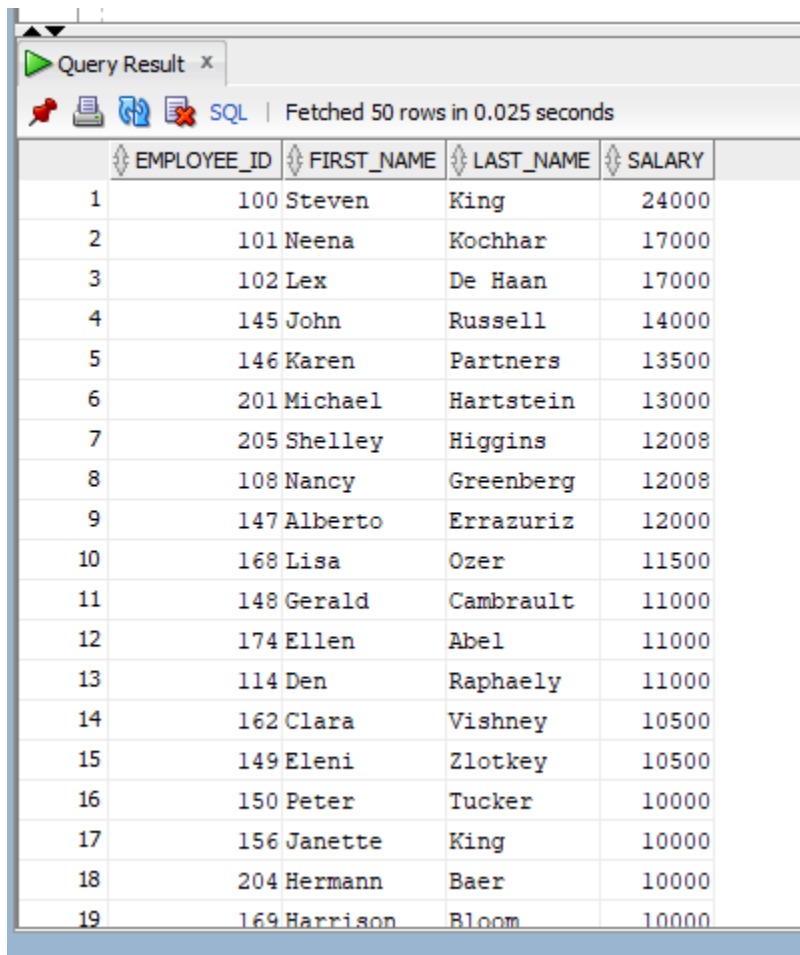
Objective

Develop analytical reports using subqueries in the Oracle HR Schema while demonstrating an understanding of business requirements, query execution, and different subquery techniques.

1. Salary Analysis Using Subqueries

Create queries for:

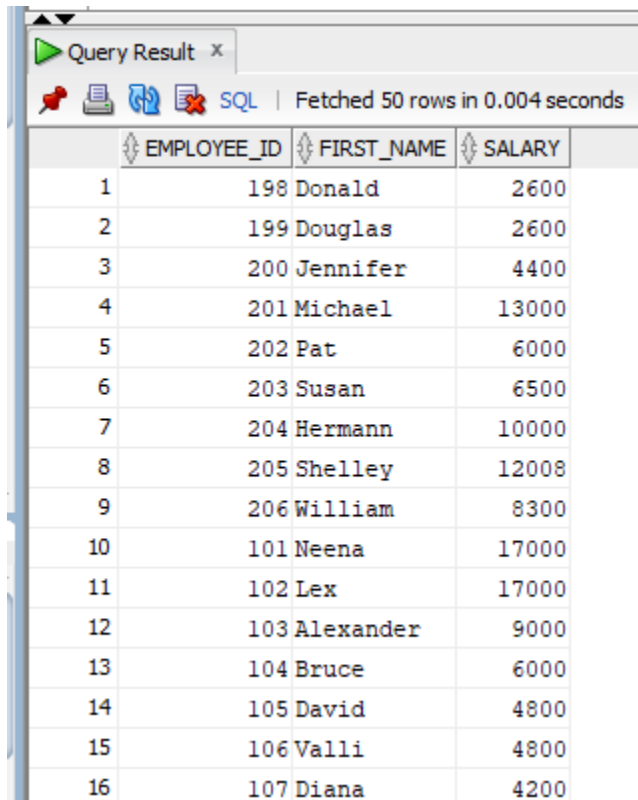
- Employees earning more than the company average salary. (Scalar Subquery)
`SELECT employee_id, first_name, last_name, salary FROM employees WHERE salary > ( SELECT AVG(salary) FROM employees) ORDER BY salary DESC;`



The screenshot shows a 'Query Result' window with a toolbar containing icons for saving, refreshing, and other database actions. Below the toolbar, it indicates 'SQL' and 'Fetched 50 rows in 0.025 seconds'. The main area displays a table with 5 columns: EMPLOYEE_ID, FIRST_NAME, LAST_NAME, and SALARY. The table lists 19 employees, sorted by salary from highest to lowest. The first employee, Steven King, has a salary of 24,000, and the last employee shown, Harrison Bloom, has a salary of 10,000.

	EMPLOYEE_ID	FIRST_NAME	LAST_NAME	SALARY
1	100	Steven	King	24000
2	101	Neena	Kochhar	17000
3	102	Lex	De Haan	17000
4	145	John	Russell	14000
5	146	Karen	Partners	13500
6	201	Michael	Hartstein	13000
7	205	Shelley	Higgins	12008
8	108	Nancy	Greenberg	12008
9	147	Alberto	Errazuriz	12000
10	168	Lisa	Ozer	11500
11	148	Gerald	Cambrault	11000
12	174	Ellen	Abel	11000
13	114	Den	Raphaely	11000
14	162	Clara	Vishney	10500
15	149	Eleni	Zlotkey	10500
16	150	Peter	Tucker	10000
17	156	Janette	King	10000
18	204	Hermann	Baer	10000
19	169	Harrison	Bloom	10000

- Employees earning less than the company's highest salary. (Scalar Subquery)
`SELECT employee_id, first_name, salary FROM employees WHERE salary < (SELECT MAX(salary) FROM employees);`



	EMPLOYEE_ID	FIRST_NAME	SALARY
1	198	Donald	2600
2	199	Douglas	2600
3	200	Jennifer	4400
4	201	Michael	13000
5	202	Pat	6000
6	203	Susan	6500
7	204	Hermann	10000
8	205	Shelley	12008
9	206	William	8300
10	101	Neena	17000
11	102	Lex	17000
12	103	Alexander	9000
13	104	Bruce	6000
14	105	David	4800
15	106	Valli	4800
16	107	Diana	4200

- Departments whose average salary exceeds the company average salary.(Scalar Subquery)

```
SELECT department_id, ROUND(AVG(salary), 2) AS avg_salary FROM employees
WHERE department_id IS NOT NULL GROUP BY department_id HAVING
AVG(salary) > (SELECT AVG(salary) FROM employees) ORDER BY avg_salary;
```

Query Result x

All Rows Fetched: 7 in 0.006 seconds

	DEPARTMENT_ID	AVG_SALARY
1	40	6500
2	100	8601.33
3	80	8955.88
4	20	9500
5	70	10000
6	110	10154
7	90	19333.33

- Employees earning the highest salary within their department. (Correlated Subquery)

```
SELECT emp.department_id, emp.employee_id, emp.first_name, emp.salary FROM
employees emp WHERE emp.salary = (SELECT MAX(s.salary) FROM employees s
WHERE s.department_id = emp.department_id ) ORDER BY emp.department_id;
```

Query Result x

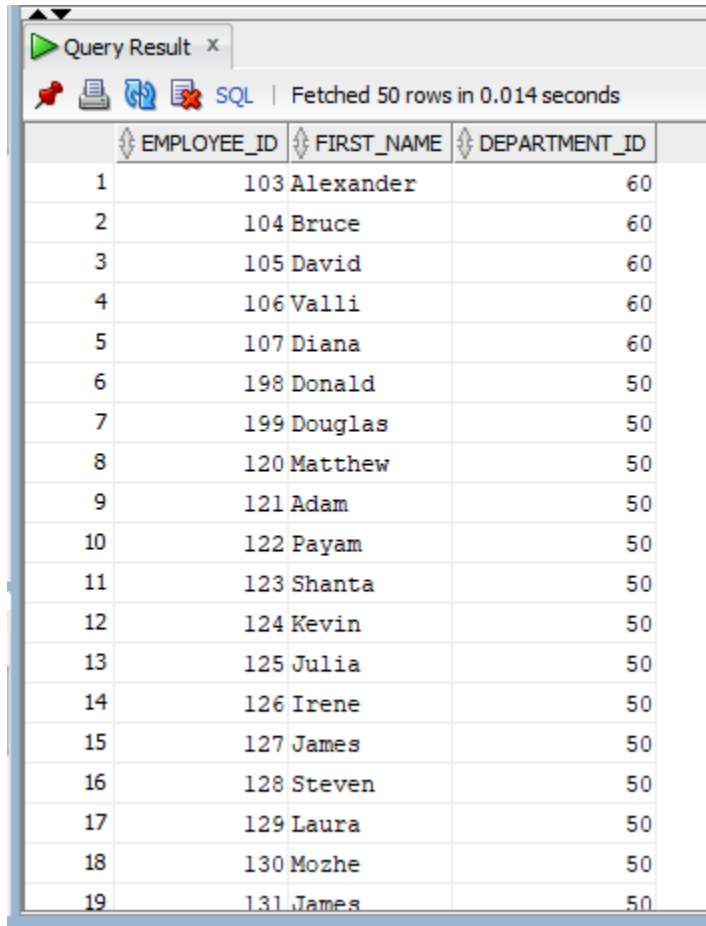
All Rows Fetched: 11 in 0.018 seconds

	DEPARTMENT_ID	EMPLOYEE_ID	FIRST_NAME	SALARY
1	10	200	Jennifer	4400
2	20	201	Michael	13000
3	30	114	Den	11000
4	40	203	Susan	6500
5	50	121	Adam	8200
6	60	103	Alexander	9000
7	70	204	Hermann	10000
8	80	145	John	14000
9	90	100	Steven	24000
10	100	108	Nancy	12008
11	110	205	Shelley	12008

2. Department & Location Analysis

Using subqueries (avoiding JOINS where required), create reports for:

- Employees working in departments located in the United States.(multi-row)
`SELECT employee_id, first_name, department_id FROM employees WHERE
department_id IN (SELECT department_id FROM departments WHERE location_id
IN (SELECT location_id FROM locations WHERE country_id = 'US'));`

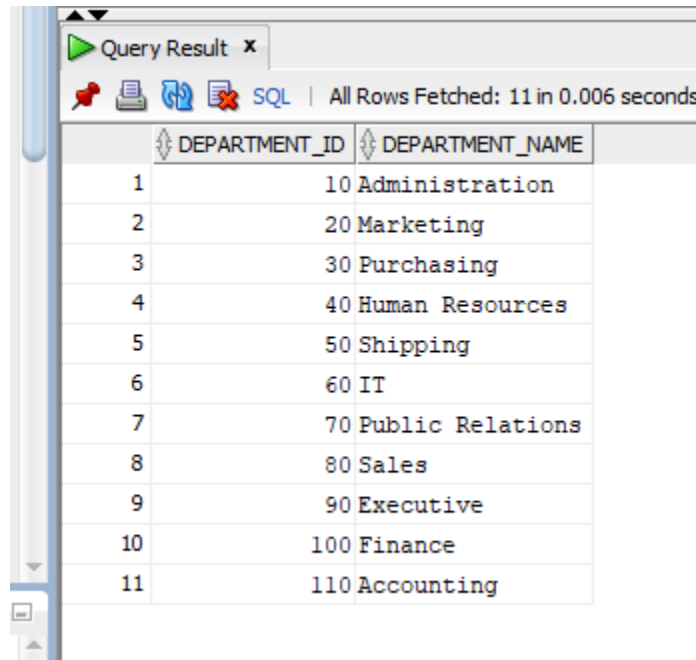


The screenshot shows a 'Query Result' window with a toolbar at the top containing icons for a pin, print, refresh, and error, along with a 'SQL' label and the text 'Fetched 50 rows in 0.014 seconds'. The table below has four columns: 'EMPLOYEE_ID', 'FIRST_NAME', and 'DEPARTMENT_ID'. The first column is not explicitly named in the header but contains row numbers 1 through 19. The data shows employees 103-107 in department 60 and employees 198-131 in department 50.

	EMPLOYEE_ID	FIRST_NAME	DEPARTMENT_ID
1	103	Alexander	60
2	104	Bruce	60
3	105	David	60
4	106	Valli	60
5	107	Diana	60
6	198	Donald	50
7	199	Douglas	50
8	120	Matthew	50
9	121	Adam	50
10	122	Payam	50
11	123	Shanta	50
12	124	Kevin	50
13	125	Julia	50
14	126	Irene	50
15	127	James	50
16	128	Steven	50
17	129	Laura	50
18	130	Mozhe	50
19	131	James	50

- Departments that currently contain employees. (Correlated Multi-Row Subquery)

```
SELECT department_id, department_name FROM departments dept WHERE EXISTS
( SELECT * FROM employees emp WHERE emp.department_id
=dept.department_id);
```



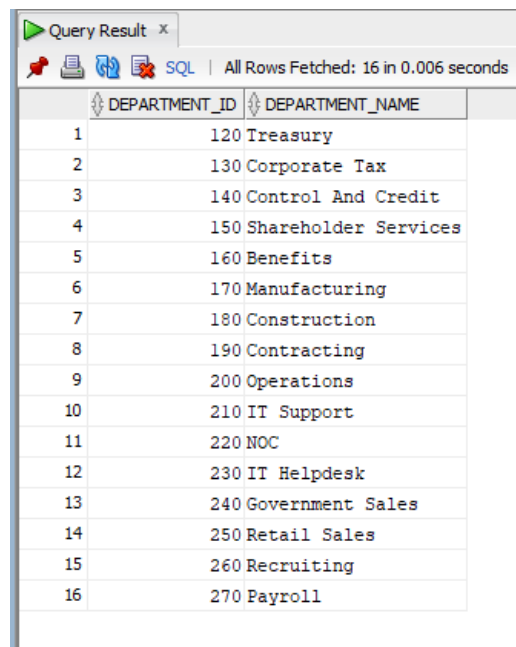
Query Result x

SQL | All Rows Fetched: 11 in 0.006 seconds

	DEPARTMENT_ID	DEPARTMENT_NAME
1	10	Administration
2	20	Marketing
3	30	Purchasing
4	40	Human Resources
5	50	Shipping
6	60	IT
7	70	Public Relations
8	80	Sales
9	90	Executive
10	100	Finance
11	110	Accounting

- Departments that currently have no employees. (Correlated Multi-Row Subquery)

```
SELECT dept.department_id, dept.department_name FROM departments dept
WHERE NOT EXISTS ( SELECT * FROM employees emp WHERE emp.department_id
= dept.department_id );
```



Query Result x

SQL | All Rows Fetched: 16 in 0.006 seconds

	DEPARTMENT_ID	DEPARTMENT_NAME
1	120	Treasury
2	130	Corporate Tax
3	140	Control And Credit
4	150	Shareholder Services
5	160	Benefits
6	170	Manufacturing
7	180	Construction
8	190	Contracting
9	200	Operations
10	210	IT Support
11	220	NOC
12	230	IT Helpdesk
13	240	Government Sales
14	250	Retail Sales
15	260	Recruiting
16	270	Payroll

- Employees working in departments located in the Europe region using nested subqueries. (Nested Multi-Row Subquery)

```
SELECT employee_id, first_name, last_name, department_id, job_id
FROM employees
WHERE department_id IN ( SELECT department_id FROM departments
WHERE location_id IN ( SELECT location_id FROM locations
WHERE country_id IN (SELECT country_id FROM countries
WHERE region_id = (SELECT region_id FROM regions
WHERE region_name = 'Europe'
)
)
)
)ORDER BY last_name, first_name;
```

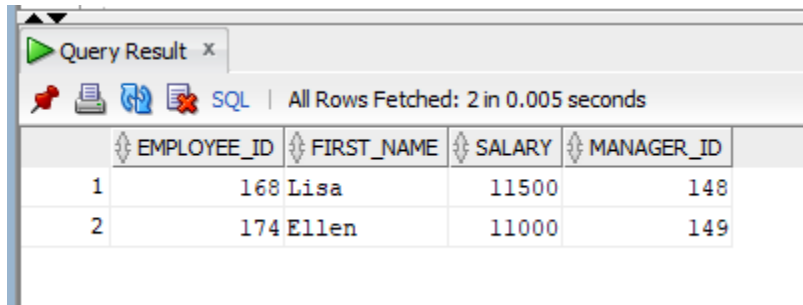
Query Result x					
All Rows Fetched: 36 in 0.031 seconds					
	EMPLOYEE_ID	FIRST_NAME	LAST_NAME	DEPARTMENT_ID	JOB_ID
1	174	Ellen	Abel	80	SA_REP
2	166	Sundar	Ande	80	SA_REP
3	204	Hermann	Baer	70	PR_REP
4	167	Amit	Banda	80	SA_REP
5	172	Elizabeth	Bates	80	SA_REP
6	151	David	Bernstein	80	SA_REP
7	169	Harrison	Bloom	80	SA_REP
8	148	Gerald	Cambrault	80	SA_MAN
9	154	Nanette	Cambrault	80	SA_REP
10	160	Louise	Doran	80	SA_REP
11	147	Alberto	Errazuriz	80	SA_MAN
12	170	Tayler	Fox	80	SA_REP
13	163	Danielle	Greene	80	SA_REP
14	152	Peter	Hall	80	SA_REP
15	175	Alyssa	Hutton	80	SA_REP
16	179	Charles	Johnson	80	SA_REP
17	156	Janette	King	80	SA_REP
18	173	Sundita	Kumar	80	SA_REP
19	165	David	Lee	80	SA_REP

3. Manager & Hierarchy Analysis

Create reports for:

- Employees earning more than their manager. (Correlated Subquery)

```
SELECT e.employee_id, e.first_name, e.salary, e.manager_id FROM employees e
WHERE e.salary > (SELECT m.salary FROM employees m WHERE m.employee_id =
e.manager_id );
```



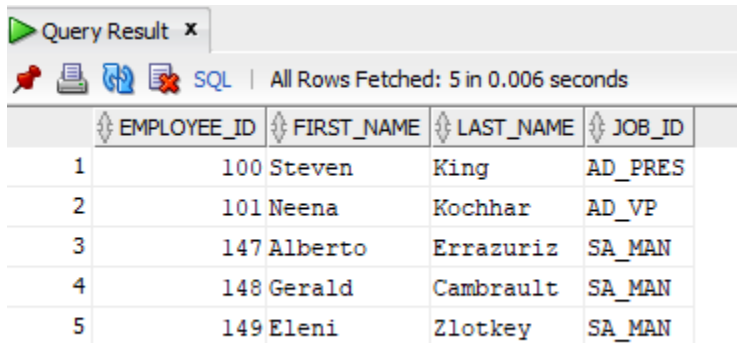
Query Result x

All Rows Fetched: 2 in 0.005 seconds

	EMPLOYEE_ID	FIRST_NAME	SALARY	MANAGER_ID
1	168	Lisa	11500	148
2	174	Ellen	11000	149

- Managers supervising at least one employee earning more than 10,000. (Multi-Row Subquery)

```
SELECT employee_id, first_name, last_name, job_id
FROM employees WHERE employee_id IN ( SELECT DISTINCT manager_id FROM
employees WHERE salary > 10000 AND manager_id IS NOT NULL );
```



Query Result x

All Rows Fetched: 5 in 0.006 seconds

	EMPLOYEE_ID	FIRST_NAME	LAST_NAME	JOB_ID
1	100	Steven	King	AD_PRES
2	101	Neena	Kochhar	AD_VP
3	147	Alberto	Errazuriz	SA_MAN
4	148	Gerald	Cambrault	SA_MAN
5	149	Eleni	Zlotkey	SA_MAN

- Employees whose manager works in the same department.

```
SELECT emp.employee_id, emp.first_name, emp.last_name, emp.department_id
FROM employees emp WHERE emp.department_id = ( SELECT m.department_id
FROM employees m WHERE m.employee_id = emp.manager_id );
```

Query Result x				
SQL Fetched 50 rows in 0.008 seconds				
	EMPLOYEE_ID	FIRST_NAME	LAST_NAME	DEPARTMENT_ID
1	198	Donald	OConnell	50
2	199	Douglas	Grant	50
3	202	Pat	Fay	20
4	206	William	Gietz	110
5	101	Neena	Kochhar	90
6	102	Lex	De Haan	90
7	104	Bruce	Ernst	60
8	105	David	Austin	60
9	106	Valli	Pataballa	60
10	107	Diana	Lorentz	60
11	109	Daniel	Faviet	100
12	110	John	Chen	100
13	111	Ismael	Sciarra	100
14	112	Jose Manuel	Urman	100
15	113	Luis	Popp	100
16	115	Alexander	Khoo	30
17	116	Shelli	Baida	30
18	117	Sigal	Tobias	30
19	118	Guy	Himuro	30

- Employees whose manager works in a different department.

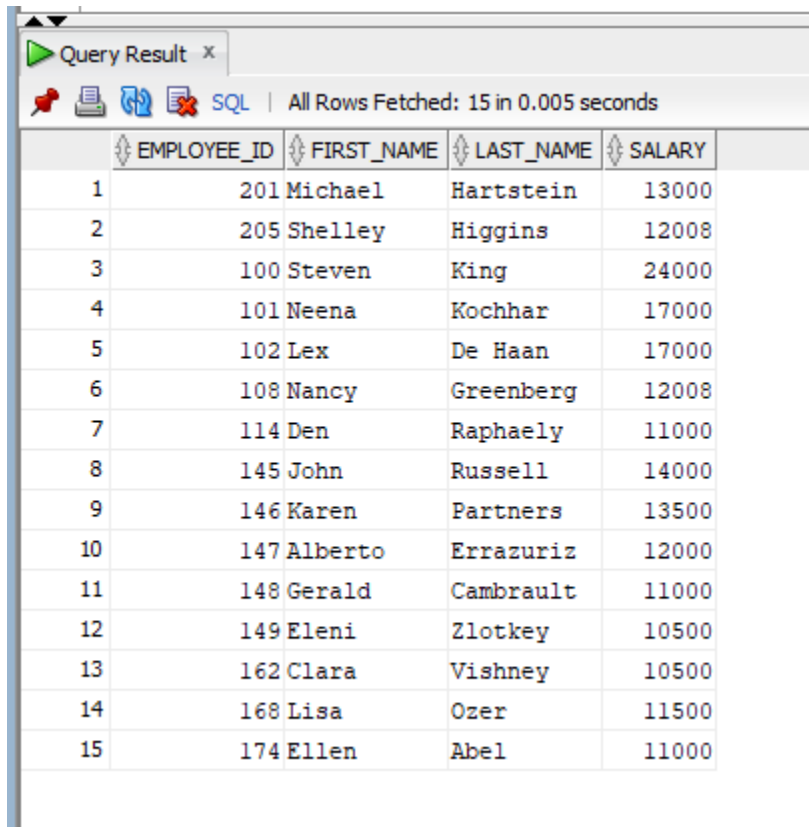
```
SELECT emp.employee_id, emp.first_name, emp.last_name, emp.department_id,
emp.manager_id FROM employees emp WHERE emp.department_id = ( SELECT
m.department_id FROM employees m WHERE m.employee_id = emp.manager_id );
```

Query Result x					
SQL All Rows Fetched: 18 in 0.003 seconds					
	EMPLOYEE_ID	FIRST_NAME	LAST_NAME	DEPARTMENT_ID	MANAGER_ID
1	200	Jennifer	Whalen	10	101
2	201	Michael	Hartstein	20	100
3	203	Susan	Mavris	40	101
4	204	Hermann	Baer	70	101
5	205	Shelley	Higgins	110	101
6	103	Alexander	Hunold	60	102
7	108	Nancy	Greenberg	100	101
8	114	Den	Raphaely	30	100
9	120	Matthew	Weiss	50	100
10	121	Adam	Fripp	50	100
11	122	Payam	Kaufling	50	100
12	123	Shanta	Vollman	50	100
13	124	Kevin	Mourgos	50	100
14	145	John	Russell	80	100
15	146	Karen	Partners	80	100
16	147	Alberto	Errazuriz	80	100
17	148	Gerald	Cambrault	80	100
18	149	Eleni	Zlotkey	80	100

4. Inline View & Query Execution

Build inline view solutions for:

- Filtering employees earning more than 10,000. (Correlated Subquery)
`SELECT emp.employee_id, first_name, last_name, emp.salary`
`FROM ( SELECT employee_id, first_name, last_name, salary FROM employees )`
`emp WHERE emp.salary > 10000;`



The screenshot shows a 'Query Result' window with a toolbar containing icons for a pin, print, copy, and SQL. The status bar indicates 'All Rows Fetched: 15 in 0.005 seconds'. The table below displays the results of the query, with columns EMPLOYEE_ID, FIRST_NAME, LAST_NAME, and SALARY. The rows are numbered 1 through 15 on the left.

	EMPLOYEE_ID	FIRST_NAME	LAST_NAME	SALARY
1	201	Michael	Hartstein	13000
2	205	Shelley	Higgins	12008
3	100	Steven	King	24000
4	101	Neena	Kochhar	17000
5	102	Lex	De Haan	17000
6	108	Nancy	Greenberg	12008
7	114	Den	Raphaely	11000
8	145	John	Russell	14000
9	146	Karen	Partners	13500
10	147	Alberto	Errazuriz	12000
11	148	Gerald	Cambrault	11000
12	149	Eleni	Zlotkey	10500
13	162	Clara	Vishney	10500
14	168	Lisa	Ozer	11500
15	174	Ellen	Abel	11000

- Displaying departments with average salary greater than 9,000.

```
SELECT d.department_id, d.department_name FROM departments d WHERE 9000
< ( SELECT AVG(e.salary) FROM employees e WHERE e.department_id =
d.department_id ) ORDER BY d.department_id;
```

Query Result x

SQL | All Rows Fetched: 4 in 0.016 seconds

	DEPARTMENT_ID	DEPARTMENT_NAME
1	20	Marketing
2	70	Public Relations
3	90	Executive
4	110	Accounting

- Displaying departments with more than five employees.

```
SELECT d.department_name, d.department_id, dept_count.emp_total
FROM departments d JOIN ( SELECT department_id, COUNT(*) AS emp_total FROM
employees GROUP BY department_id ) dept_count ON d.department_id =
dept_count.department_id WHERE dept_count.emp_total > 5;
```

Query Result x

SQL | All Rows Fetched: 4 in 0.006 seconds

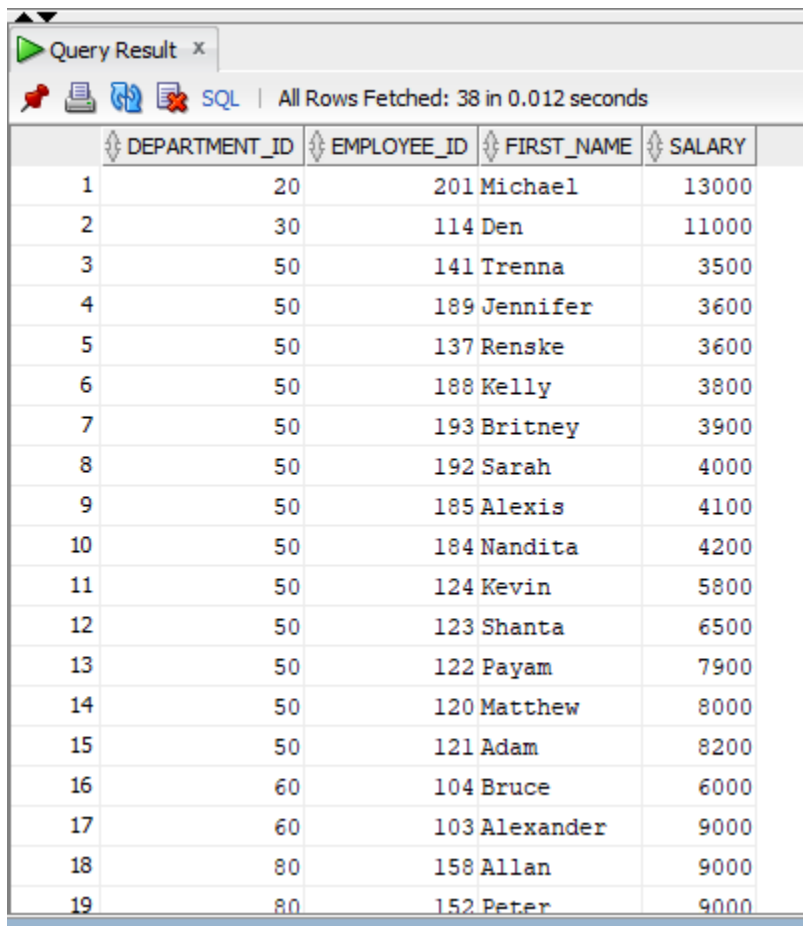
	DEPARTMENT_NAME	DEPARTMENT_ID	EMP_TOTAL
1	Sales	80	34
2	Finance	100	6
3	Shipping	50	45
4	Purchasing	30	6

5. Advanced Subquery Logic

Create reports for:

- Employees earning above their department average salary.

```
SELECT emp.department_id, emp.employee_id, emp.first_name, emp.salary FROM  
employees emp WHERE emp.salary > ( SELECT AVG(s.salary) FROM employees s  
WHERE s.department_id = emp.department_id ) ORDER BY emp.department_id;
```



The screenshot shows a SQL query result window titled "Query Result". It displays 19 rows of data from the employees table, ordered by department_id. The columns are DEPARTMENT_ID, EMPLOYEE_ID, FIRST_NAME, and SALARY. The data is as follows:

	DEPARTMENT_ID	EMPLOYEE_ID	FIRST_NAME	SALARY
1	20	201	Michael	13000
2	30	114	Den	11000
3	50	141	Trenna	3500
4	50	189	Jennifer	3600
5	50	137	Renske	3600
6	50	188	Kelly	3800
7	50	193	Britney	3900
8	50	192	Sarah	4000
9	50	185	Alexis	4100
10	50	184	Nandita	4200
11	50	124	Kevin	5800
12	50	123	Shanta	6500
13	50	122	Payam	7900
14	50	120	Matthew	8000
15	50	121	Adam	8200
16	60	104	Bruce	6000
17	60	103	Alexander	9000
18	80	158	Allan	9000
19	80	152	Peter	9000

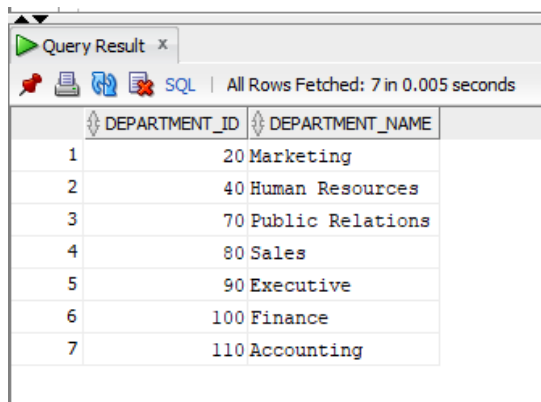
- Employees earning above the average salary for their job role.

```
SELECT emp.employee_id, emp.job_id, emp.first_name, emp.salary
FROM employees emp WHERE emp.salary > ( SELECT AVG(j.salary)
FROM employees j WHERE j.job_id = emp.job_id ) ORDER BY emp.job_id;
```

Query Result x				
SQL All Rows Fetched: 43 in 0.012 seconds				
	EMPLOYEE_ID	JOB_ID	FIRST_NAME	SALARY
1	110	FI_ACCOUNT	John	8200
2	109	FI_ACCOUNT	Daniel	9000
3	104	IT_PROG	Bruce	6000
4	103	IT_PROG	Alexander	9000
5	117	PU_CLERK	Sigal	2800
6	116	PU_CLERK	Shelli	2900
7	115	PU_CLERK	Alexander	3100
8	146	SA_MAN	Karen	13500
9	145	SA_MAN	John	14000
10	177	SA_REP	Jack	8400
11	176	SA_REP	Jonathon	8600
12	175	SA_REP	Alyssa	8800
13	158	SA_REP	Allan	9000
14	152	SA_REP	Peter	9000
15	163	SA_REP	Danielle	9500
16	151	SA_REP	David	9500
17	157	SA_REP	Patrick	9500
18	170	SA_REP	Tayler	9600
19	169	SA_REP	Harrison	10000

- Departments where all employees earn more than 5,000.

```
SELECT dept.department_id, dept.department_name FROM departments dept
WHERE EXISTS ( SELECT * FROM employees emp WHERE emp.department_id =
dept.department_id ) AND NOT EXISTS ( SELECT * FROM employees e WHERE
e.department_id = dept.department_id AND e.salary <= 5000 );
```



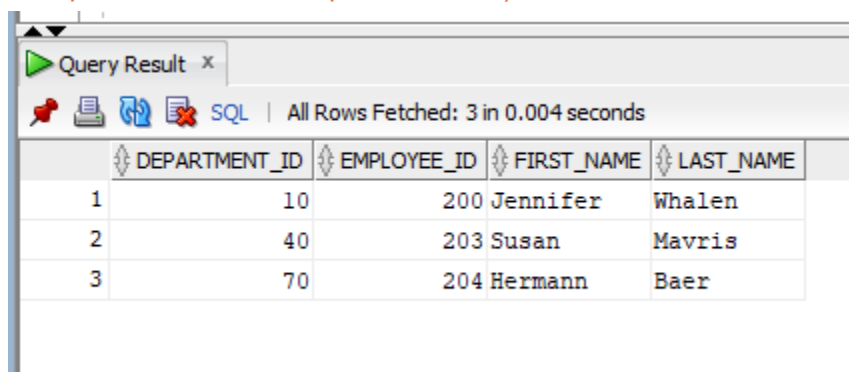
Query Result x

All Rows Fetched: 7 in 0.005 seconds

	DEPARTMENT_ID	DEPARTMENT_NAME
1	20	Marketing
2	40	Human Resources
3	70	Public Relations
4	80	Sales
5	90	Executive
6	100	Finance
7	110	Accounting

- Employees who are the only employee in their department.

```
SELECT e.department_id, e.employee_id, e.first_name, e.last_name
FROM employees e WHERE ( SELECT COUNT(*) FROM employees c WHERE
c.department_id = e.department_id ) = 1;
```



Query Result x

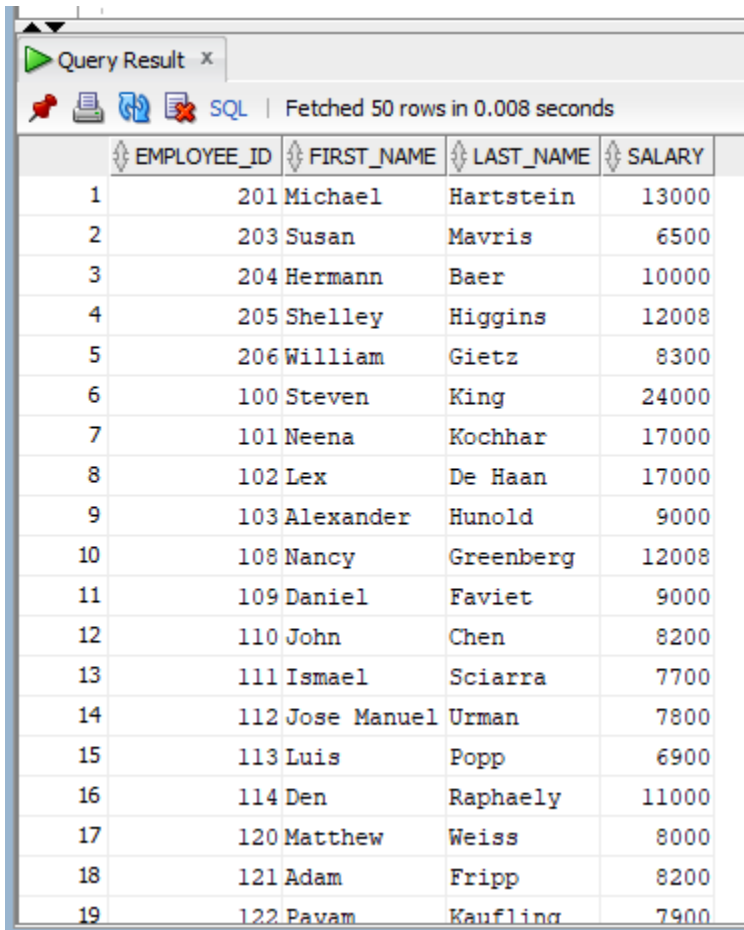
All Rows Fetched: 3 in 0.004 seconds

	DEPARTMENT_ID	EMPLOYEE_ID	FIRST_NAME	LAST_NAME
1	10	200	Jennifer	Whalen
2	40	203	Susan	Mavris
3	70	204	Hermann	Baer

6. Executive HR Analytics Challenge

Develop analytical reports using subqueries for:

- Employees earning above the company average salary. (Scalar Subquery)
`SELECT employee_id, first_name, last_name, salary FROM employees WHERE salary > (SELECT AVG(salary) FROM employees);`

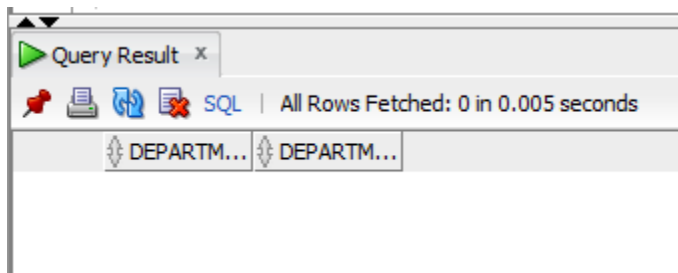


The screenshot shows a 'Query Result' window with a toolbar at the top containing icons for a pin, print, refresh, and error, along with a status bar indicating 'SQL | Fetched 50 rows in 0.008 seconds'. The table below displays the results of the SQL query, listing 19 employees with their IDs, first names, last names, and salaries. The salaries are sorted in descending order.

	EMPLOYEE_ID	FIRST_NAME	LAST_NAME	SALARY
1	201	Michael	Hartstein	13000
2	203	Susan	Mavris	6500
3	204	Hermann	Baer	10000
4	205	Shelley	Higgins	12008
5	206	William	Gietz	8300
6	100	Steven	King	24000
7	101	Neena	Kochhar	17000
8	102	Lex	De Haan	17000
9	103	Alexander	Hunold	9000
10	108	Nancy	Greenberg	12008
11	109	Daniel	Faviet	9000
12	110	John	Chen	8200
13	111	Ismael	Sciarra	7700
14	112	Jose Manuel	Urman	7800
15	113	Luis	Popp	6900
16	114	Den	Raphaely	11000
17	120	Matthew	Weiss	8000
18	121	Adam	Fripp	8200
19	122	Pavam	Kaufling	7900

- Departments with more employees than Department 50.

```
SELECT d.department_id, d.department_name FROM departments d
WHERE ( SELECT COUNT(*) FROM employees e WHERE e.department_id =
d.department_id ) > ( SELECT COUNT(*) FROM employees WHERE department_id =
50 ) ORDER BY d.department_id;
```



Query Result x

All Rows Fetched: 0 in 0.005 seconds

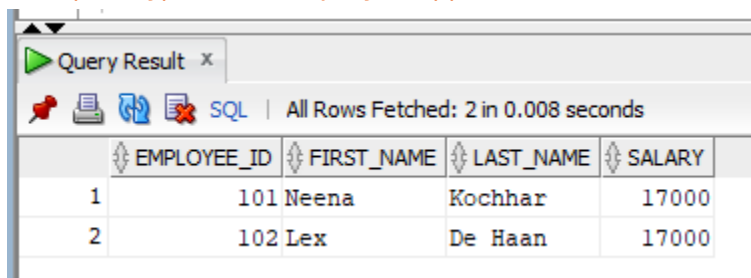
DEPARTM...	DEPARTM...
------------	------------

- Employees working in the department with the highest average salary.

```
SELECT employee_id, first_name, last_name, department_id, salary FROM
employees WHERE department_id = ( SELECT department_id FROM ( SELECT
department_id, AVG(salary) AS avg_sal FROM employees WHERE department_id IS
NOT NULL GROUP BY department_id ORDER BY avg_sal DESC ) WHERE ROWNUM
= 1 );
```

- Employees earning the second-highest salary.

```
SELECT employee_id, first_name, last_name, salary FROM employees
WHERE salary = ( SELECT MAX(salary) FROM employees WHERE salary < ( SELECT
MAX(salary) FROM employees ) );
```



Query Result x

All Rows Fetched: 2 in 0.008 seconds

	EMPLOYEE_ID	FIRST_NAME	LAST_NAME	SALARY
1	101	Neena	Kochhar	17000
2	102	Lex	De Haan	17000

- Managers whose employees' average salary exceeds 10,000.

SELECT m.employee_id, m.first_name, m.last_name FROM employees m WHERE
m.employee_id IN (SELECT manager_id FROM employees WHERE manager_id IS
NOT NULL GROUP BY manager_id HAVING AVG(salary) > 10000);

```
SELECT m.employee_id, m.first_name, m.last_name
FROM employees m
WHERE m.employee_id IN (
  SELECT manager_id
  FROM employees
  WHERE manager_id IS NOT NULL
  GROUP BY manager_id
  HAVING AVG(salary) > 10000
);
```